Name- Harshi jain | Branch- B.tech cse |

Roll no.- 21 | Section- A

Labsheet-9

QUESTIONS

Question 1. Aim

Design, develop and test a full-stack CRUD Student Record Management web application integrating frontend, REST API and database.

Question 2. Objectives

Connect frontend, backend and database; implement REST APIs and database CRUD; consume APIs using fetch/async-await; validate form data; and test APIs with Postman.

Question 3. Software / Tools Required

HTML5, CSS3, JavaScript, Node.js, Express.js, MongoDB, Mongoose, Postman/Thunder Client, VS Code and Chrome/Firefox.

Question 4. Problem Statement

Build a Student Record Management System with a form for Name, Roll No., Course and Marks; REST APIs for add/view/update/delete; persistent database storage; a table with Edit/Delete actions; and validation including Marks 0–100.

Question 5. REST API Endpoints

POST /students — Add student
GET /students — Fetch all students
GET /students/:id — Fetch one student
PUT /students/:id — Update student
DELETE /students/:id — Delete student

Question 6. Project Structure

```
StudentRecordManagement/
├── models/Student.js
├── routes/studentRoutes.js
├── public/index.html
├── public/style.css
├── public/script.js
└── server.js
```

FULL CODE

Create the files shown by the comments below and paste the complete code.

```
// ================== server.js ==================
const express=require("express"),mongoose=require("mongoose"),cors=require("cors"),path=require("path");
const routes=require("./routes/studentRoutes");
const app=express();
app.use(cors());app.use(express.json());app.use(express.static("public"));
mongoose.connect("mongodb://127.0.0.1:27017/studentDB")
.then(()=>console.log("MongoDB connected")).catch(console.error);
app.use("/students",routes);
app.listen(5000,()=>console.log("http://localhost:5000"));


// ================== models/Student.js ==================
const mongoose=require("mongoose");
const schema=new mongoose.Schema({
 name:{type:String,required:true},
 rollNo:{type:String,required:true,unique:true},
 course:{type:String,required:true},
```

```
 marks:{type:Number,required:true,min:0,max:100}
});
module.exports=mongoose.model("Student",schema);


// ================= routes/studentRoutes.js =================
const router=require("express").Router();
const Student=require("../models/Student");

router.post("/",async(req,res)=>{
 try{res.status(201).json(await Student.create(req.body))}
 catch(e){res.status(400).json({message:e.message})}
});

router.get("/",async(req,res)=>{
 try{res.json(await Student.find())}
 catch(e){res.status(500).json({message:e.message})}
});

router.get("/:id",async(req,res)=>{
 try{
  const s=await Student.findById(req.params.id);
  if(!s)return res.status(404).json({message:"Student not found"});
  res.json(s);
 }catch(e){res.status(400).json({message:"Invalid ID"})}
});

router.put("/:id",async(req,res)=>{
 try{
  const s=await Student.findByIdAndUpdate(req.params.id,req.body,{new:true,runValidators:true});
  if(!s)return res.status(404).json({message:"Student not found"});
  res.json(s);
 }catch(e){res.status(400).json({message:e.message})}
});

router.delete("/:id",async(req,res)=>{
 try{
  const s=await Student.findByIdAndDelete(req.params.id);
  if(!s)return res.status(404).json({message:"Student not found"});
  res.json({message:"Student deleted"});
 }catch(e){res.status(400).json({message:"Invalid ID"})}
});
module.exports=router;


// ================= public/index.html =================
<!DOCTYPE html>
<html>
<head>
<title>Student Records</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1>Student Record Management</h1>
<form id="form">
<input id="id" type="hidden">
```

```
<input id="name" placeholder="Name" required>
<input id="rollNo" placeholder="Roll No." required>
<input id="course" placeholder="Course" required>
<input id="marks" type="number" min="0" max="100" placeholder="Marks" required>
<button>Add / Update</button>
</form>
<table>
<thead><tr><th>Name</th><th>Roll No.</th><th>Course</th><th>Marks</th><th>Action</th></tr></thead>
<tbody id="list"></tbody>
</table>
<script src="script.js"></script>
</body>
</html>

// ================== public/style.css ==================
body{font-family:Arial;margin:30px;background:#f5f5f5}
input,button{padding:8px;margin:4px}
table{width:100%;border-collapse:collapse;margin-top:20px;background:white}
th,td{padding:8px;border:1px solid #ccc;text-align:center}

// ================== public/script.js ==================
const f=document.getElementById("form"),list=document.getElementById("list");
const fields=["name","rollNo","course","marks"];

async function load(){
 const d=await(await fetch("/students")).json();

list.innerHTML=d.map(s=>`<tr><td>${s.name}</td><td>${s.rollNo}</td><td>${s.course}</td><td>${s.marks}</t
d><td><button onclick="edit('${s._id}')">Edit</button><button
onclick="del('${s._id}')">Delete</button></td></tr>`).join("");
}
f.onsubmit=async e=>{
 e.preventDefault();
 const s=Object.fromEntries(fields.map(x=>[x,document.getElementById(x).value]));
 s.marks=Number(s.marks);
 if(!s.name||!s.rollNo||!s.course||s.marks<0||s.marks>100)return alert("Enter valid data");
 const id=document.getElementById("id").value;
 await fetch("/students"+(id?"/"+id:""),{method:id?"PUT":"POST",headers:{"Content-
Type":"application/json"},body:JSON.stringify(s)});
 f.reset();document.getElementById("id").value="";load();
};
async function edit(id){
 const s=await(await fetch("/students/"+id)).json();
 document.getElementById("id").value=s._id;
 fields.forEach(x=>document.getElementById(x).value=s[x]);
}
async function del(id){
 if(confirm("Delete student?")){await fetch("/students/"+id,{method:"DELETE"});load();}
}
load();
```

RUN COMMANDS

```
npm init -y
npm install express mongoose cors
npm install --save-dev nodemon
npm run dev
```

Output

Student Record Management System

Name

sarthak jain

Roll No.

51

Course

Btech cse

Marks

95

Add Student

Student Records

Name	Roll No.	Course	Marks	Actions

Student Record Management System

Name

Enter student name

Roll No.

Enter roll number

Course

Enter course

Marks

Enter marks

Add Student

Student added successfully

Student Records

Name	Roll No.	Course	Marks	Actions
sarthak jain	51	Btech cse	95	Edit Delete

Student Record Management System

Name

sarthak jain

Roll No.

51

Course

Btech cse

Marks

85

Update Student

Cancel

Name

Enter student name

Roll No.

Enter roll number

Course

Enter course

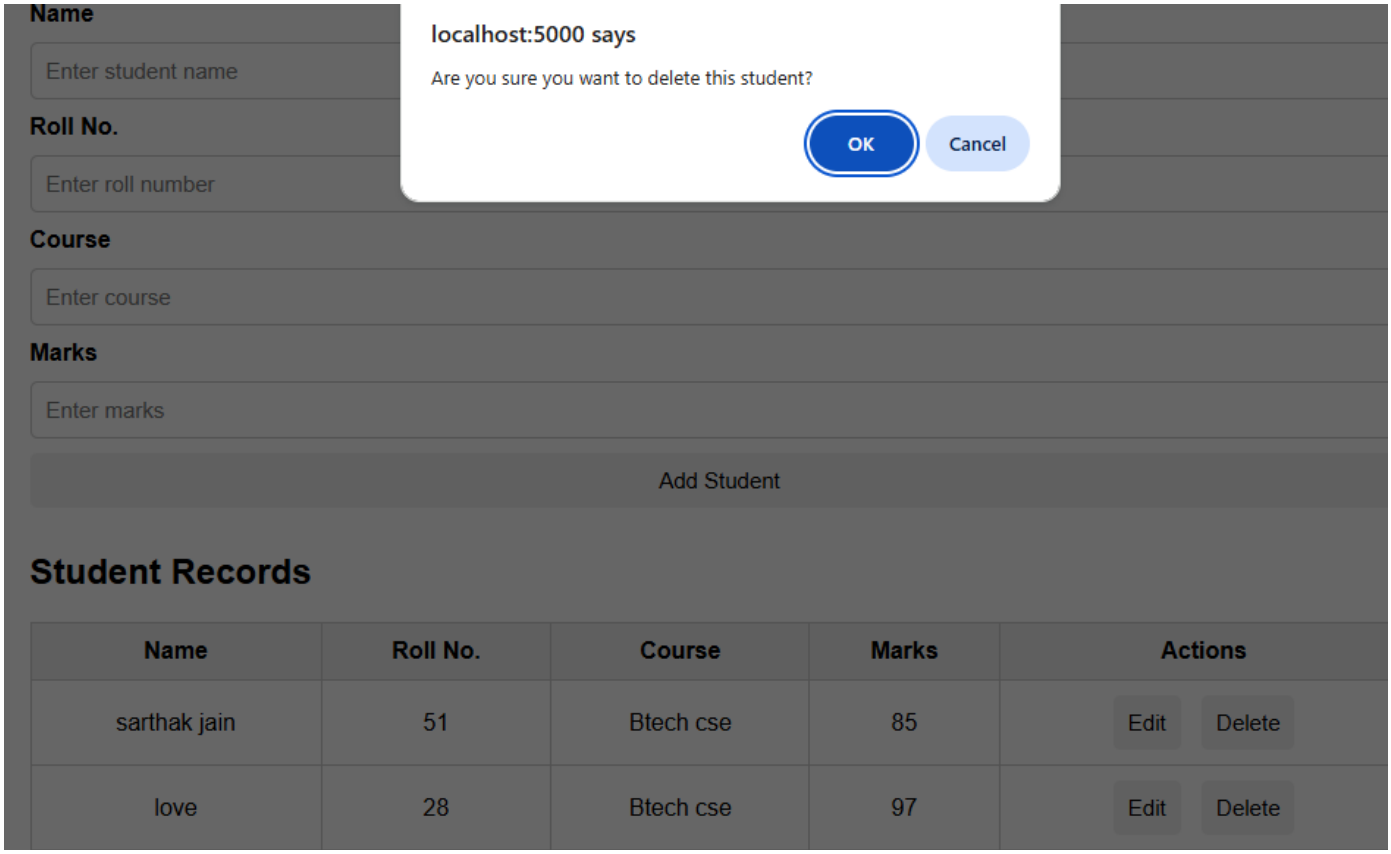
Marks

Enter marks

Add Student

Student Records

Name	Roll No.	Course	Marks	Actions
sarthak jain	51	Btech cse	85	Edit Delete
love	28	Btech cse	97	Edit Delete



Name

Enter student name

Roll No.

Enter roll number

Course

Enter course

Marks

Enter marks

Add Student

Student Records

Name	Roll No.	Course	Marks	Actions
sarthak jain	51	Btech cse	85	Edit Delete